MODERATED LEARNING MANAGEMENT SYSTEM (LMS)
Software Engineering Project Report | CS4433
Katleho Ramaili | Student number: 202322674 | Submission: 3 November 2026
Repository: https://github.com/amira2k5/Moderated_LMS_Software_Engineering

ABSTRACT

This report documents the engineering of a Moderated Learning Management System (LMS) for a secondary school, from feasibility and project planning through requirements, design, implementation, verification, release and post-release maintenance. The system addresses a specific institutional need: managing courses, assignments, student submissions, marking and assessment records while ensuring that a student cannot view a mark until the relevant head of department (HoD) has checked and released it. Four account roles are supported: student, lecturer, HoD and administrator. The project was developed as an offline-capable C++17 console application using SQLite for persistence, libsodium for password hashing and GoogleTest for automated testing. The work followed a staged waterfall process with three review gates. This approach was selected because the assignment required defined deliverables and formal review before implementation, while the requirements and design needed to be sufficiently stable before coding began (Royce, 1970; Sommerville, 2016).

The delivered version 1.0.0 was subsequently improved through change request CR1, which introduced class-wide deadline extensions and a display of extended deadlines on student assignment lists. The resulting version 1.1.0 retained the existing database schema and late-penalty logic. The report records 104 automated tests, all passing, and 98.6% line coverage for the business-logic layer. The measured login time was approximately 78–80 milliseconds with 800 student and 60 staff accounts stored, comfortably below the stated two-second target. These results are evidence from the project's recorded local test environment; they are not a claim that the software has been validated on every school computer or through a hosted continuous-integration run.

The central engineering outcome is an explicit moderation workflow. A lecturer may mark a submission, but the mark remains unavailable to the student while it is in the Marked or Moderated state. The HoD reviews and approves marks, then performs a separate release action. Mark changes require a reason and are written to an append-only audit log. The report also identifies limitations rather than presenting the project as complete in every operational respect: real-lecturer usability testing, testing on the school's older computers and a native Windows build, an on-site backup restoration rehearsal, encryption at rest, automated retention and learner-data export remain outstanding. These limitations inform the recommended next steps.

1. INTRODUCTION AND PROJECT CONTEXT

The school requires a single system to coordinate the academic activities that connect courses, assignments, submissions and assessment. Before a centralised system, these activities can be distributed across paper records, individual files and informal communication. That arrangement can make it difficult to establish which submission is current, whether an extension was granted, whether a penalty was applied consistently, and whether marks were approved before being communicated. A software system can improve consistency, but only if its rules are clear and its access controls reflect the school's responsibilities.

The project therefore treats moderation as the defining business requirement rather than as an optional administrative feature. Students may submit work and later view released results. Lecturers create assignments, inspect submissions, provide marks and written feedback, and grant individual extensions. HoDs review departmental marks, adjust or correct a mark when justified, approve a batch and release approved marks. Administrators establish accounts, departments, courses, enrolments and lecturer assignments, and perform backups. The system supports these activities without requiring an internet connection and is intended for one school computer or a small local-network installation. The design targets 800 stored student accounts, 60 staff accounts and up to five simultaneous sessions.

The project excludes online payments, video conferencing, automated grading, plagiarism detection and web or mobile interfaces. These exclusions are important because they prevent scope expansion from undermining the core workflow within a fixed seven-week schedule. The first release uses a menu-driven console because it is comparatively inexpensive to build, works with keyboard input and can run on modest hardware. A graphical interface may improve ease of use, but it was not necessary to demonstrate the principal business rules in the first release.

The engineering objective was to take the system through the full software life cycle and produce evidence for each major decision. The work includes a feasibility study and risk register; a requirements specification and use-case model; architecture, class, behavioural and database design; implementation; verification and validation; configuration management and quality assurance; deployment planning; and maintenance following a change request. The waterfall model was used with deliverables at phase boundaries and three review gates. The intention was not to suggest that change would never occur, but to make decisions visible and controlled. The CR1 change demonstrates that the baseline could be amended through impact analysis, implementation and regression testing.

The report uses the project's own recorded figures and terminology. Where the original project evidence distinguishes between a measured result and an unverified assumption, that distinction is retained. For example, the local performance measurements are reported, but the report does not claim that the system has passed a school-site acceptance test. This distinction is consistent with professional responsibility: a report should communicate what was built and tested, as well as what remains uncertain (ACM, 2018; BCS, 2022).

2. PROJECT MANAGEMENT, FEASIBILITY AND RISK

The feasibility study considered technical, operational, economic, legal and schedule feasibility. Technically, C++17, SQLite, CMake, GoogleTest and libsodium provide established tools that can run without a database server or continuous internet access. SQLite is embedded, which reduces installation and administration requirements for a school with limited infrastructure. The anticipated number of accounts and assessment records was considered modest enough for the selected storage approach. The principal technical risk was not that the technologies were unsuitable, but that implementation would require the developer to work across C++, persistence, testing and security libraries within a short period.

Operationally, a role-specific menu was selected. Students need a short route to courses, assignments, submission and released feedback; lecturers need assignment creation, extension and marking functions; HoDs need review, approval, release and audit functions; administrators need account and course administration. Separating menus reduces the chance that a user will be overwhelmed by options unrelated to their work. However, a console interface is not automatically usable merely because it is simple to implement. The requirement for first-time lecturers to create an assignment within five minutes was defined as measurable, but it remains unverified because real lecturers were not included in a formal usability study.

Economic feasibility was favourable because the selected dependencies are available without commercial licence fees and the system can use existing school hardware. The main cost is development effort, estimated at approximately 245 hours over seven weeks. Legal feasibility required careful treatment of learner information and assessment records. The design therefore limits collected information, applies role-based access, stores password hashes rather than clear-text passwords and records mark changes. The project identifies the Data Protection Act 2011 of Lesotho as relevant legislation, while acknowledging that the school should obtain appropriate legal advice about its current obligations (Kingdom of Lesotho, 2011).

Schedule feasibility was the weakest dimension. The deadline was fixed, and the work had dependencies: stakeholder decisions informed the SRS; the SRS informed the use cases; the design had to be reviewed before implementation; and testing depended on working code. MoSCoW prioritisation was used to protect Must requirements. The optional CSV export was identified as the first feature to drop if the schedule became threatened. This decision protected the central moderation workflow and essential security controls.

The work breakdown structure followed nine life-cycle branches: planning, requirements, design, implementation, verification and validation, configuration management and quality assurance, deployment, maintenance and completion. Twenty-seven tasks were linked through finish-to-start dependencies. Three phase gates served as milestones. Gate 1 reviewed the SRS and use-case model; Gate 2 reviewed the design document; Gate 3 reviewed test evidence and the tagged release. Implementation was not to begin before Gate 2. The critical path ran through stakeholder analysis, requirements, use cases, architecture, class design, repository setup, domain implementation, unit and integration testing, system testing and release. It then continued through CR1 analysis, implementation, regression testing and final reporting. Some documentation and supporting tasks had between 0.1 and 0.7 weeks of slack. The schedule contained estimates rather than timesheet-confirmed actual effort, so the report does not present estimated hours as measured labour.

Risk management began with eight risks covering technical, project and people concerns. Likelihood and impact were each scored from one to five, and exposure was calculated as their product. The highest initial exposure was schedule overrun (20), followed by inconsistent resolution of stakeholder conflicts (16), skill gaps in C++, SQLite and continuous integration (12), and database loss or corruption (10). Mitigations included prioritising Must requirements, recording stakeholder resolutions, using small interfaces and early tests, and providing transactions, backups and a restore guide. The register was rescored at each gate. At completion, five of eleven risks were closed and six remained open with residual exposure between six and nine. Outstanding concerns include the absence of real-user usability testing, unverified operation on older school computers, limitations of SQLite on a network folder, an untested native Windows build and other accepted limitations. Keeping these risks visible is preferable to

describing them as resolved when no supporting test exists.

3. REQUIREMENTS ENGINEERING

Eight stakeholder groups were identified: the Principal, lecturers, HoDs, students, the student representative, the administrator, the IT officer, and indirect stakeholders such as parents and the data-protection regulator. Five interview statements were used as the principal source of requirements. They were incomplete and sometimes contradictory, so eleven conflicts or ambiguities were recorded and resolved before the requirements baseline was approved. This process follows the principle that requirements should be explicit, traceable and verifiable rather than left as informal interpretations (ISO/IEC/IEEE, 2018).

The most significant conflict concerned mark visibility. Lecturers preferred students to see marks immediately after marking, while the Principal and HoD required departmental checking before release. The resolution gives priority to the moderation requirement: students see a mark only when its state is Released. To reduce delay, the HoD can approve and release an entire assignment batch. A second important ambiguity concerned late penalties. The agreed rule applies a deduction of 10% of the assignment's maximum mark for every started 24-hour period, up to three periods; after that, the final mark is zero. The calculation uses the student's effective deadline, including any extension. Late work is still accepted and flagged, but cannot be replaced after the deadline. Marking opens only after the effective deadline, and written feedback is required.

Other resolutions define how the system behaves in less central situations. A student may resubmit before the effective deadline while the submission remains in the Submitted state; the latest version is marked, while earlier versions are retained. The lecturer cannot change a mark after HoD approval through the ordinary lecturer re-mark path. A lecturer may change a mark before approval, while the HoD may adjust a mark before release and correct it after release, provided a reason is recorded. Each account has exactly one role. The system's performance and usability statements were converted into measurable targets: login within two seconds for the specified stored population, and first-time assignment creation within five minutes. The account scale was clarified as 800 stored students and up to five simultaneous sessions.

The SRS was organised around an introduction, overall description, business rules and specific requirements. It contained 23 functional requirements in version 1.0: 18 Must, four Should and one Could. CR1 added FR-24. It also contained 12 non-functional requirements: eight Must and four Should. Each requirement has an identifier, priority, source and verification method. This makes it possible to trace a stakeholder concern through design and implementation to a test. The business rules BR-1 to BR-10 define effective deadlines, late-day calculation, penalties, submission states, version retention, resubmission, marking windows, role restrictions and the requirement for reasons when marks are changed.

Examples of non-functional requirements illustrate the approach. NFR-01 sets a login target of two seconds with 800 students and 60 staff stored. NFR-04 requires salted Argon2id password hashes. NFR-06 requires at least 50 invalid inputs to be rejected without a crash. NFR-07 requires every mark change to create an audit entry that cannot be edited. NFR-11 sets a minimum 70% line-coverage target for business logic. These statements are testable because the project identifies a test, inspection, demonstration or measurement for each one. The traceability matrix in Appendix A links the requirements to use cases, design elements, source files and test names.

MoSCoW prioritisation was used to manage scope. The moderation rule and functions necessary for the school's core assessment workflow were Must requirements. Functions that strengthen operation, including password change and lockout, audit viewing, backup and correction of released marks, were prioritised as Should. CSV export was the single Could requirement and was not implemented. Payment, conferencing, automated grading, plagiarism detection and web/mobile interfaces remained outside the release scope. The use-case model contains 21 initial use cases for the four actors, which specialise an abstract User actor. Marking includes late-penalty application and audit recording; late submission and resubmission extend the submission use case; and mark adjustment extends moderation. CR1 introduced a twenty-second use case, Grant class extension.

Three use cases were fully dressed: Submit assignment, Mark submission, and Moderate and release marks. Their specifications contain preconditions, main flows, alternatives, exceptions, postconditions and business rules, with 31 flows in total. The submission use case validates enrolment and the selected file, including existence, readability, non-empty content and a maximum size of 5 MB. It calculates the effective deadline, stores an on-time file as a version, and preserves earlier versions if a permitted resubmission occurs. A first submission after the deadline is accepted in the Late state. Exceptions include a non-enrolled student, invalid file, prohibited replacement, an already marked submission and storage failure. A storage failure must not leave a partially saved submission. These flows were used as the basis for system tests.

4. SOFTWARE DESIGN AND DATABASE

The architecture is a layered design packaged as one program. The presentation layer is a console interface with role-specific menus. The application layer contains services corresponding to groups of use cases. The domain layer contains entities, role permissions, the late-penalty policy and mark-change events. The data-access layer contains repository interfaces and SQLite implementations. Dependencies point downward, and SQL statements are confined to the data-access layer. This arrangement supports the offline requirement and makes business rules testable without depending on a live database.

A single-process layered architecture was chosen instead of a client-server design because the IT requirement favours offline operation on older computers and does not require a database server. Model-View-Controller was considered but rejected as the principal architecture: its separation of input and display is useful for event-driven graphical interfaces, but it does not by itself define persistence boundaries or the location of business rules. The selected structure keeps password hashing and SQL in clearly identified components. Services depend on interfaces, allowing tests to replace SQLite repositories with in-memory fakes and the system clock with a controlled clock. This is especially important for testing deadline boundaries reproducibly.

The domain model uses an abstract User class with Student, Lecturer, HeadOfDepartment and Administrator subclasses. Each subclass implements allows(Action), so permission decisions are associated with the role rather than scattered as repeated role comparisons throughout the application. A Submission owns its versions and at most one Mark. The design uses four patterns, selected for concrete reasons (Gamma et al., 1994; Fowler, 2002). Strategy encapsulates the late-penalty calculation in ILatePenaltyPolicy, implemented by TenPercentPerDayPolicy. This isolates a rule likely to change and allows tests to substitute a different policy without rewriting marking. Repository places persistence behind interfaces and confines SQL to the data layer. Factory creates the appropriate User subclass from a stored role and enforces the requirement that an HoD has a department. Observer publishes mark-change events so AuditObserver can write the corresponding audit record within the same transaction.

The design also applies the SOLID principles (Martin, 2003). Services have focused responsibilities. New penalty policies, observers and roles can be introduced through additional classes rather than extensive modification of existing logic. User subclasses preserve the contract of the base class. Repository interfaces are kept small, and services receive dependencies through constructors. The main function is responsible for creating concrete SQLite and libsodium implementations. These choices support testability and reduce coupling, although they do not remove the need for integration tests against the real persistence layer.

The late-penalty Strategy receives the maximum mark, raw mark, effective deadline and submission time. If the submission is not later than the effective deadline, days late is zero. Otherwise, the number of started 24-hour periods is calculated. For one to three periods, 10% of the maximum mark is deducted per period; the final mark cannot be negative. More than three periods results in zero. For an assignment worth 20 and a raw mark of 16, submission at the deadline retains 16. One second late produces 14; exactly 24 hours late also produces 14; 24 hours and one second late produces 12; exactly 72 hours late produces 10; and 72 hours and one second late produces zero. These boundary examples were translated into automated tests so that the implementation follows the written business rule precisely.

The submission life cycle is represented by five states: Submitted or Late on arrival, followed by Marked, Moderated and Released. A Late submission cannot return to Submitted, enforcing the rule that late work cannot be replaced. The state model also represents permitted re-marking and correction of released marks, with guards and recorded reasons. An activity model describes the end-to-end process across student, lecturer, HoD and system swimlanes. Sequence diagrams describe submission and moderation/release interactions. In particular, approval and release are separate transactions, preventing a batch from being applied only partially.

The database contains eleven tables and is designed in third normal form. Each column stores a single value. Submission versions are stored in a separate table rather than as repeated columns. Many-to-many relationships, such as students to courses and lecturers to courses, are represented by association tables with composite keys. Department names are stored once, avoiding transitive dependencies. The current submission version and penalty are derived rather than duplicated. The mark record stores days_late and final_mark because they preserve the assessment outcome calculated at marking time; later changes to a policy should not silently rewrite historical results.

The four user types share user_account with a role field and constraints, because they share most account attributes and login requires one lookup. A check constraint ensures that only an HoD account has a department association. Enumerated states are stored as text and protected by CHECK constraints. Some rules span tables and cannot be expressed through a simple table constraint—for example, that a student must be enrolled in the course associated with an assignment. Such rules are enforced in services and covered by tests. Append-only triggers prevent audit_log rows from being updated or deleted. The design therefore combines database constraints for local integrity with service-level validation for business rules that involve multiple records.

5. IMPLEMENTATION, SECURITY AND PERSISTENCE

Implementation follows the approved design and is organised into 14 source files under src/, with 2,304 non-comment lines recorded in the project. The files cover domain objects and patterns, the SQLite wrapper and unit of work, repositories, password hashing, seven application services, the console interface and the program entry point. CMake is used for the build, with -Wall, -Wextra and -Wpedantic enabled. The coding standard defines naming, layering, SQL, password and error-handling rules. The recorded build completed without compiler warnings.

Service methods receive collaborators through interfaces. This means that the same business logic can run with SQLite in the deployed program and with fakes in tests. An IUnitOfWork groups multi-row operations into a transaction that rolls back unless committed. It supports nesting, so a service can call another service while preserving the atomicity of the overall operation. Batch release and the audit record therefore succeed or fail together. Time is stored in UTC as ISO 8601 text and represented as seconds in memory; conversion to local time occurs at the console boundary. This keeps deadline arithmetic independent of the computer's local time-zone display. Penalties use integer arithmetic on seconds, and marks are rounded half up to two decimal places.

The console uses table-driven menus and a common loop that catches LmsError and displays a readable message. Validation, access-denied, rule-violation, not-found and database errors share this hierarchy. Invalid input should produce a recoverable message rather than terminate the program. Testability is supported by replaceable seams: a Clock allows tests to set exact times; a file-store interface allows simulated storage failure; and observers can be configured to throw, demonstrating that a mark must not be saved if its audit entry cannot be written. These mechanisms test failure paths without introducing special test-only branches into production code.

Passwords are stored as salted Argon2id hashes generated by libsodium; the system does not store clear-text passwords. Argon2id is specified in RFC 9106 and is designed to make password guessing more expensive through memory-hard computation (Biryukov et al., 2021). OWASP recommends Argon2id for new password-storage implementations (OWASP, n.d.). Using a maintained cryptographic library avoids writing a password-hashing algorithm manually. The library generates a random salt for each password and stores the parameters with the hash, allowing the work factor to be raised in future. The recorded interactive parameters are 64 MiB of memory and two passes. The measured login time was approximately 78–80 ms, below the two-second requirement. A test confirms that the same password produces different hashes for two users and that the stored value does not contain the clear password.

Role-based access control is enforced in service methods rather than relying only on hidden menu options. Each operation checks the caller's permissions and limits lecturers and HoDs to their assigned courses or departments. Accounts are locked for five minutes after five consecutive incorrect password attempts. Unknown usernames and incorrect passwords produce the same message, reducing username discovery through the login screen. SQL statements are prepared and bound rather than assembled from user text, reducing SQL-injection risk (OWASP, n.d.). The project records a test using the string x'; DROP TABLE user_account; -- as a username, which was stored as text without damaging the database.

SQLite stores data in one database file and is accessed through repositories. Foreign-key enforcement is enabled, the busy timeout is five seconds and writes use BEGIN IMMEDIATE. These settings support controlled access when up to five sessions share the database file, although the project still identifies a network-folder deployment as a limitation requiring operational caution. Submitted files are copied to an application folder under generated names. The database stores the path, size and original filename. The administrator can create a backup through SQLite's online backup interface. The installation guidance recommends a weekly backup to a USB drive or a second computer.

Every mark change records the actor, time, old value, new value and reason. This applies to initial marking, re-marking, HoD adjustment and correction after release. The audit log is protected by append-only database triggers. This provides accountability and supports investigation, but it should not be confused with a fully tamper-proof security architecture: a person with direct control over the host computer and database files may have powers beyond those of an ordinary LMS account. Operating-system permissions and physical protection are therefore part of deployment guidance. Encryption at rest is not implemented and remains a known limitation.

The implementation differs from the design in a small number of documented ways. AssignmentService uses mark and user repositories and, after CR1, the unit of work. AdminService holds the database object for backup. The initial simple transaction helper was replaced by IUnitOfWork, and the file store became an interface. The database schema remains consistent with the design document. Recording these differences makes the implementation auditable and prevents the report from claiming a perfect one-to-one correspondence where the code evolved during development.

6. VERIFICATION, QUALITY, DEPLOYMENT AND MAINTENANCE

Verification and validation followed the V-Model. Unit tests verify detailed design; integration tests verify architectural interactions; system tests verify the SRS and use-case specifications; and acceptance scenarios relate the system to stakeholder concerns. Each level has entry and exit criteria. Unit testing begins after warning-free compilation and exits when all tests pass and business-logic coverage is at least 70%. Integration testing begins after unit tests pass and exits with no open high-severity defect. System testing begins after integration tests pass and requires coverage of every flow in the three detailed use cases, together with non-functional target evidence. Acceptance testing demonstrates stakeholder concerns or explicitly records them as unverified.

Test design uses equivalence partitioning and boundary-value analysis. Login classes include valid users, unknown users, wrong passwords, inactive accounts and locked accounts. Submission classes include enrolled and non-enrolled students; valid, missing, empty and oversized files; and the states of an existing submission. Mark tests include values within range, at the boundaries and outside the permitted range, as well as present and blank feedback. Deadline boundaries receive particular attention: one second before, exactly at and one second after the deadline; exactly 24 hours and 24 hours plus one second; exactly 72 hours and 72 hours plus one second. This is necessary because the started-24-hour rule produces different outcomes at precise boundaries.

The recorded test suite contains 104 tests: nine unit tests, 46 integration tests, 39 system tests and ten CR1 tests. All 104 passed. gcov reported 98.6% line coverage for the business-logic layer, representing 546 of 554 lines, against the 70% target. Overall coverage excluding the entry point was 92.5%. The few uncovered business-logic lines were defensive branches. The test environment uses a fixed clock, in-memory database and simulated failing file store or observer to exercise error paths. Coverage is useful evidence of executed code, but it does not prove that every possible behaviour is correct; the test cases and their relationship to requirements remain important.

System tests cover both normal and exceptional use-case flows. Examples include an un-enrolled student attempting to submit, empty or oversized files, attempts to replace late work, marks outside the permitted range, database errors during marking and a batch release that fails part-way through. Tests confirm that a database failure does not leave a partial mark or audit entry. Robustness tests submit 60 invalid inputs to services and confirm that they are rejected with readable messages without crashing. Acceptance scenarios demonstrate the Principal's moderation requirement, lecturer extensions, the HoD's penalty and review responsibilities, student resubmission and feedback rules, and the IT officer's offline and performance concerns. The requirement for simplicity with non-IT lecturers remains unverified because a real-user study has not been performed.

Performance measurements used a database containing 800 student and 60 staff accounts. Login took 78 ms in the acceptance summary (approximately 80 ms in the implementation discussion); listing 150 submissions took 0.5 ms; saving a mark took 2.4 ms; and a class extension for 800 students took 32 ms. These are below the project's one- to two-second targets. A concurrency test using five database connections performed 100 writes without failure, and the database integrity check passed. The measurements are specific to the test environment and should be repeated on the school's actual hardware before operational acceptance.

Seven defects were logged. DEF-01, a medium-severity design gap, was discovered when the extension screen required a way to list students in a course; studentsIn() was added and tested. DEF-02 was an incorrect expected value in a test. DEF-03 was a missing include in the test build. DEF-04 identified functions above the complexity limit, including menus with complexity up to 23; table-driven menus and extracted validators reduced complexity. DEF-05 was a CR1 regression in which inserting a menu option renumbered existing choices; moving the new option to position seven restored compatibility, and all 93 original tests passed. DEF-06 covered static-analysis style findings, all corrected. DEF-07 identified the absence of an automated test for the concurrency requirement; a concurrency test was added and passed. No high-severity defect remained open, and no defect was found in the core penalty or moderation flow.

Configuration management used Git and a GitHub-flow-inspired branching approach. The main branch was intended to remain buildable and test-passing. Work was completed on short-lived feature branches and merged through pull requests with a review checklist. The repository recorded 24 commits, nine merged pull requests and two tagged releases, v1.0.0 and v1.1.0. Semantic versioning communicates the scale of a change: the initial release is 1.0.0, CR1 increases the minor number to 1.1.0 because it adds functionality, and a corrective-only release would increment the patch number. The GitHub Actions workflow defines build/test and static-analysis/coverage jobs. However, the hosted workflow runs are not claimed as evidence; the same commands were executed locally. This distinction is retained to avoid overstating the CI evidence.

Quality assurance included cppcheck and lizard. cppcheck initially found style problems in five groups, which were corrected; the recorded final analysis

reported no findings for source and tests. Lizard measured 2,304 lines of code and 213 functions, with an average of seven lines and average cyclomatic complexity of 2.5. Seven functions initially exceeded the standard's complexity limit of ten. The menus were refactored into table-driven structures and validators were extracted. Two functions remained at complexity 13 and 11; these were accepted as straightforward sequences of tested conditions. A twelve-point self-review examined SQL construction, layer boundaries, password handling, role checks, transaction atomicity, resource release, error messages, function size, naming, constructors, design fidelity and duplication. The review led to concrete code improvements rather than serving only as a checklist.

Deployment targets one school computer running Ubuntu 22.04 or later, or Windows 10 with WSL2. The program, SQLite file and submitted-file folder reside on that computer alongside the required libraries. A local-network folder is described as an optional light-use arrangement for up to five sessions, but network-folder SQLite behaviour remains a risk and should be confirmed in the school environment. The technician's installation guide estimates approximately thirty minutes for installing tools, building and copying the program, creating the first administrator and checking setup by creating a department. The guide also covers weekly backup, a ten-minute restore procedure, updates and common error messages. Before go-live, the technician should create departments and accounts, enrol students, assign lecturers, run a rehearsal assignment through submission, marking, moderation and release, take a backup and rehearse restoration. User manuals are provided for all four roles.

After release, CR1 requested a class-wide deadline extension and visibility of the revised deadline on student assignment lists. This is classified as perfective maintenance because it improves a functioning system rather than correcting a fault, adapting to a changed environment or restructuring to prevent a future problem (ISO/IEC/IEEE, 2022). Impact analysis identified changes to FR-10 and FR-06, the addition of FR-24, a new use case and an alternative flow, updates to four classes, a new result type and a new menu item. The database schema did not need to change.

The selected implementation stores one extension row for each enrolled student. A separate class-wide extension table could automatically cover students who enrol later, but it would require a schema change and migration and would require the effective-deadline calculation to consult two places. The per-student approach reuses the existing extension mechanism and keeps the penalty code unchanged. Its limitation is that students who enrol after the class extension is granted are not automatically included; this is documented in the manual and risk register. The grantClassExtension operation updates all applicable rows in one transaction, never shortens a later extension already held by a student, and reports how many students were changed or retained a later date.

CR1 was implemented on a separate branch and released as v1.1.0. The student assignment list displays an "extended to" date where applicable, and the lecturer menu includes the class-wide extension option. All 93 tests from v1.0.0 were rerun unchanged and passed; ten new tests cover class-wide updates, preservation of later extensions, penalty effects, validation, access control, transaction atomicity, student display and console interaction. The first version of the menu change caused DEF-05 by renumbering existing options, demonstrating why regression testing matters. After correction, all tests passed and the extension for 800 students measured 32 ms. Release notes, the user manual, traceability matrix and risk register were updated.

7. PROFESSIONAL, LEGAL AND ETHICAL CONSIDERATIONS; CONCLUSION

The LMS stores names, account details, submissions, marks and feedback relating to learners, many of whom are children. The project identifies the Data Protection Act 2011 of Lesotho as relevant and recognises principles including lawful and specific purpose, data minimisation, accuracy, appropriate security, limited retention and individuals' ability to access or correct their information (Kingdom of Lesotho, 2011). The design collects only information needed for assessment and does not store addresses, identity numbers or contact details. Students can access only their own work and released marks; lecturers are limited to their courses; and HoDs are limited to their departments. Audit records support accountability by recording reasons for changes.

Two data-protection capabilities are not implemented in version 1.1: automated deletion after a retention period and a learner-data export facility. CSV export was not implemented, and deletion/retention tools remain future work. Until these functions are provided, the school would need to manage requests through the administrator and database file, subject to its legal obligations and appropriate procedures. The report does not claim that the software alone guarantees legal compliance. The school must determine its lawful basis, retention schedule, notices, access procedures and safeguards with appropriate advice.

Confidentiality, integrity and availability are addressed in different ways. Role-based access and the release rule protect confidentiality of assessment results. Password hashing and generic login errors strengthen account security. Prepared SQL statements reduce injection risk. Transactions, reasoned mark changes and the append-only audit log support integrity. Backup functionality and a restore guide support availability. However, the database and submitted files are not encrypted at rest. Someone with direct access to the computer's files may therefore read them. The installation guidance recommends operating-system permissions and physical security, while encryption at rest is identified as future work. A single administrator account also has substantial authority and should be carefully controlled.

Accessibility has both strengths and limitations. The console is keyboard-operated and does not rely on colour to communicate status; it uses words such as LATE and RELEASED. Terminal font size can be increased, and terminal screen readers may provide access for some users. Nevertheless, the system has not been tested with a screen reader or with people with disabilities. Terminal accessibility varies across platforms, and a learner who cannot easily type a file path may need assistance. A graphical file chooser and testing with representative users are sensible future improvements. Accessibility should be evaluated with actual users rather than inferred from the presence of keyboard input.

The late-penalty rule is transparent and consistently applied, but consistency does not automatically make a policy fair in every circumstance. A submission one second late incurs a full started-day penalty, and the transition from a 30% deduction to zero after 72 hours is abrupt. Weekends count as time. These effects were made explicit in stakeholder resolutions and manuals. Students are informed of the number of late periods, and lecturers and HoDs can see the penalty before release. Individual and class-wide extensions provide a way to respond to illness, disruption or events such as a power cut. HoDs can adjust or correct marks with a reason, and the Strategy pattern allows the school to change the policy later without rewriting marking logic. Late work is accepted and can still receive feedback, even where the final mark is zero. Human oversight remains important because an automated calculation can apply a rule consistently while the rule itself may still require review.

Professional responsibility includes honesty about evidence, attention to the people affected by the system and respect for the school's operating constraints. The report identifies the unverified usability requirement, untested platforms and hardware, absent retention and export tools, and lack of encryption at rest. Dependencies were selected with school use and licence conditions in mind. Stakeholder conflicts were resolved in writing so that policy decisions can be reviewed. These practices align with professional expectations to avoid harm, act competently, respect privacy and be transparent about limitations (ACM, 2018; BCS, 2022).

In conclusion, the project delivered a working Moderated LMS and documented its progression through planning, requirements, design, implementation, testing, quality assurance, deployment and maintenance. The most important requirement —that students must not see a mark before HoD release—is reflected in the domain model, service permissions, state transitions and acceptance tests. Mark changes require reasons and are recorded in an append-only log. The layered architecture, interfaces, replaceable clock and unit of work made business rules testable and helped the regression suite detect a real menu compatibility problem during CR1. The stakeholder decision log reduced ambiguity, while the traceability matrix connected requirements to implementation and evidence.

The measured test results are positive: 104 automated tests passed, business-logic coverage reached 98.6%, and key operations were substantially below their response-time targets in the local test environment. These results support the conclusion that the implemented core behaves as specified under the tested conditions. They do not remove the need for school-site validation. The next priorities should be usability testing with lecturers and learners, installation and performance tests on the actual school computers, a witnessed backup restoration, encryption at rest, data-retention and export processes, and a review of the late-penalty policy. A later release could also provide a graphical interface, automatically include later-enrolled students in class extensions and make the penalty policy configurable. The project is therefore a tested and maintainable first operational version, with clearly identified work required before broad institutional deployment.

REFERENCES (HARVARD STYLE)

ACM (2018) *ACM Code of Ethics and Professional Conduct*. New York: Association for Computing Machinery.

BCS (2022) *Code of Conduct for BCS Members*. Swindon: BCS, The Chartered Institute for IT.

Biryukov, A., Dinu, D., Khovratovich, D. and Josefsson, S. (2021) *Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications*. RFC 9106. Internet Engineering Task Force. Available at: https://www.rfc-editor.org/rfc/rfc9106 (Accessed: 3 November 2026).

Denis, F. (n.d.) *Libsodium documentation: password hashing*. Available at: https://doc.libsodium.org (Accessed: 3 November 2026).

Fowler, M. (2002) *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.

ISO/IEC/IEEE (2018) *ISO/IEC/IEEE 29148:2018 Systems and software

engineering—Life cycle processes—Requirements engineering*. Geneva: International Organization for Standardization.

ISO/IEC/IEEE (2022) *ISO/IEC/IEEE 14764:2022 Software engineering—Software life cycle processes—Maintenance*. Geneva: International Organization for Standardization.

Kingdom of Lesotho (2011) *Data Protection Act, 2011*. Maseru: Government of Lesotho.

Martin, R.C. (2003) *Agile Software Development: Principles, Patterns, and Practices*. Upper Saddle River, NJ: Prentice Hall.

OWASP (n.d.) *Password Storage Cheat Sheet* and *SQL Injection Prevention Cheat Sheet*. OWASP Cheat Sheet Series. Available at: https://cheatsheetseries.owasp.org (Accessed: 3 November 2026).

Royce, W.W. (1970) ‘Managing the development of large software systems’, *Proceedings of IEEE WESCON*, pp. 1–9.

SQLite Consortium (n.d.) *SQLite documentation*. Available at: https://www.sqlite.org/docs.html (Accessed: 3 November 2026).

Sommerville, I. (2016) *Software Engineering*. 10th edn. Harlow: Pearson.